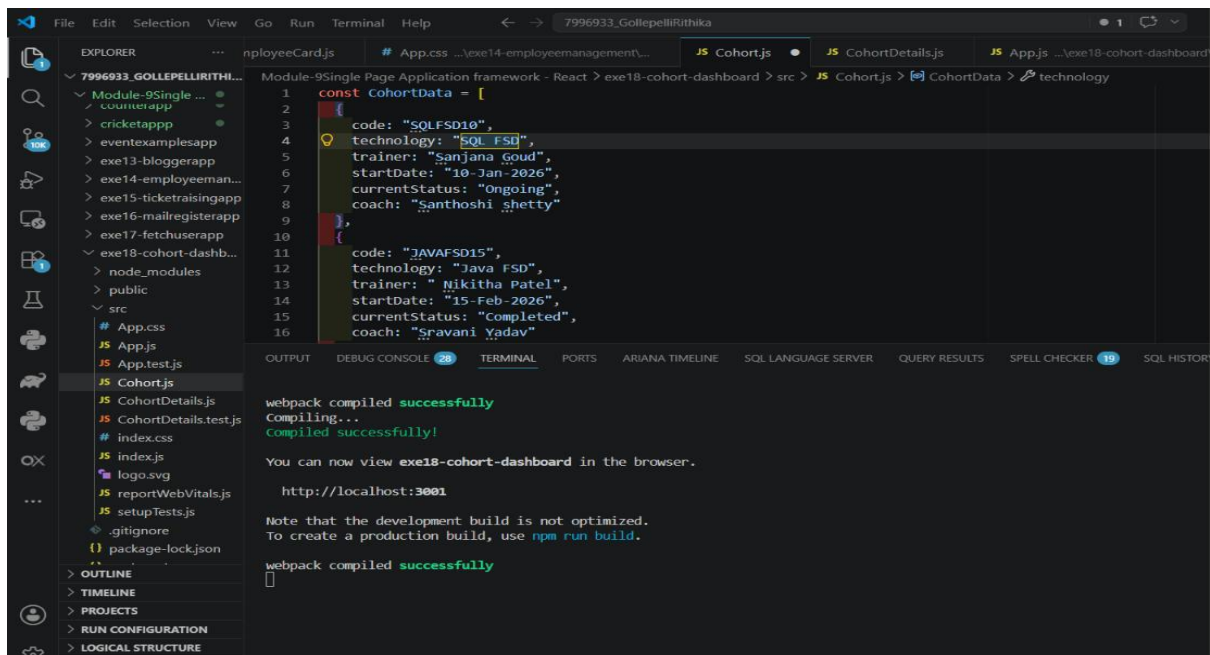


Output18Exe18CohortDashboard.ReactJS-HOL



The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `Cohort.js` file with the following code:

```
1 const CohortData = [
2   {
3     code: "SQLFSD10",
4     technology: "SQL FSD",
5     trainer: "Sanjana Goud",
6     startDate: "10-Jan-2026",
7     currentStatus: "Ongoing",
8     coach: "Santhoshi shetty"
9   },
10  {
11    code: "JAVAFSD15",
12    technology: "Java FSD",
13    trainer: "Nikitha Patel",
14    startDate: "15-Feb-2026",
15    currentStatus: "Completed",
16    coach: "Sravani Yadav"
17  }
18 ]
```

The terminal at the bottom shows the following output:

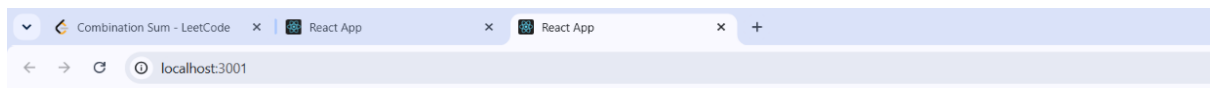
```
webpack compiled successfully
Compiling...
compiled successfully!

You can now view exe18-cohort-dashboard in the browser.

http://localhost:3001

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```



Cohort Dashboard

SQLFSD10

Technology: SQL FSD
Trainer: Sanjana Goud
Start Date: 10-Jan-2026
Status: Ongoing
Coach: Santhoshi shetty

JAVAFSD15

Technology: Java FSD
Trainer: Nikitha Patel
Start Date: 15-Feb-2026
Status: Completed
Coach: Sravani Yadav

REACT20

Technology: React
Trainer: Vineela Netha
Start Date: 20-May-2026
Status: Ongoing
Coach: Dharani Shetty

The application displays the Cohort Dashboard, which contains information about different training cohorts. Each cohort is shown inside a separate card with details such as the cohort code, technology, trainer name, start date, current status, and coach name.

Objectives

1. Explain the Need for Unit Testing in React

Unit Testing is the process of testing individual components, functions, or modules of a React application independently to ensure they work correctly. Each unit is tested in isolation without depending on other parts of the application, making it easier to identify and fix bugs early in the development process.

Need for Unit Testing

- Detects bugs at an early stage.
- Ensures each component works as expected.
- Improves code quality and reliability.
- Simplifies debugging and maintenance.

Benefits

- Faster identification of errors.
- Increases confidence when modifying code.
- Improves application stability. .

2. Working with Jest and Enzyme in React

Jest

Jest is a JavaScript testing framework developed by Meta (Facebook). It is the most widely used testing framework for React applications and is used for unit testing, mocking, snapshot testing, and generating test coverage reports.

Features of Jest

- Easy to configure.
- Supports unit testing and integration testing.
- Built-in mocking functionality.
- Snapshot testing for React components.

Enzyme

Enzyme is a React testing utility developed by Airbnb that helps developers test React components by rendering them and interacting with their output. It allows testing of component behavior without running the complete application.

Enzyme provides three rendering methods:

- **Shallow Rendering** – Tests only the selected component without rendering its child components.
- **Full DOM Rendering (mount)** – Renders the entire component tree for comprehensive testing.
- **Static Rendering (render)** – Produces static HTML output for simple rendering checks.

Benefits of Jest and Enzyme

- Simplifies testing of React components.
- Detects UI issues before deployment.
- Reduces debugging time.
- Improves application reliability.

3. List the Types of Router Components

React Router provides different router components to manage navigation within a React application.

Router Component	Description
BrowserRouter	Uses the browser's History API and is the most commonly used router for web applications.
HashRouter	Uses the URL hash (#) for routing and is useful for static hosting environments.
MemoryRouter	Stores navigation history in memory and is mainly used for testing or non-browser environments.
StaticRouter	Used for server-side rendering (SSR) applications.